

Rapport Technique Complet



Type de projet : Embarqué / Robotique / Vision par Ordinateur

Matériels : Arduino Uno, Raspberry Pi, L298N, Webcam

Langages : Python 3, C++ (Arduino), protocole MQTT

Bibliothèques : OpenCV, MediaPipe, Paho-MQTT, PySerial

Réalisé par : IVOMBO Glenn Silvère

Module : Traitement d'Image

Filière : Ingénierie Robotique & Cobotique

Niveau : 1^{ère} Année

Année universitaire : 2024 – 2025

Table des matières

1	Introduction et Présentation du Projet	3
1.1	Contexte et Motivation	3
1.2	Architecture Générale du Système	3
1.3	Gestes Reconnus	4
2	Matériel Nécessaire	5
2.1	Liste du Matériel	5
2.2	Schéma de Câblage Arduino – L298N – Moteurs	6
3	Installation des Logiciels et Dépendances	7
3.1	Environnement côté Ordinateur / Raspberry Pi	7
3.1.1	Python 3 et pip	7
3.1.2	Création d'un environnement virtuel (recommandé)	7
3.1.3	Bibliothèques Python requises	7
3.2	Installation du Broker MQTT (Mosquitto)	8
3.3	Environnement côté Arduino	8
3.3.1	Identifier le port série sous Linux	8
4	Code Source Commenté	10
4.1	Sketch Arduino : Contrôle des Moteurs (<code>motors.ino</code>)	10
4.2	Script Python : Pont MQTT – Série (<code>bridge.py</code>)	11
4.3	Script Python : Détection de Gestes (<code>vision.py</code>)	12
4.3.1	Configuration et paramètres	12
4.3.2	Détection de l'état des doigts	12
4.3.3	Boucle principale de capture et d'envoi	13
5	Flux de Données et Protocoles	16
5.1	Le Protocole MQTT	16
5.2	Communication Série (UART)	16
6	Mise en Œuvre et Démarrage	17
6.1	Ordre de Démarrage	17
6.2	Résolution des Problèmes Courants	18
7	Pistes d'Amélioration	19
7.1	Améliorations Fonctionnelles	19
7.2	Améliorations Techniques	19

Chapitre 1

Introduction et Présentation du Projet

1.1 Contexte et Motivation

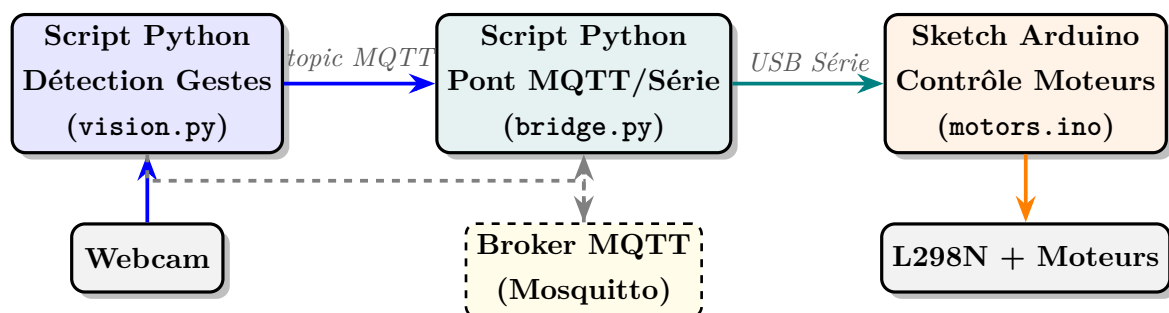
Ce projet propose la réalisation d'une **voiture télécommandée sans contact**, pilotée uniquement par des gestes de la main captés via une webcam. L'objectif est de combiner plusieurs domaines de l'informatique embarquée et de l'intelligence artificielle :

- la **vision par ordinateur** (détection de la main en temps réel) ;
- la **communication réseau légère** (protocole MQTT) ;
- la **programmation embarquée** (contrôle de moteurs avec Arduino).

Le résultat est un système complet, de bout en bout, où un geste de la main se traduit en mouvement physique d'un véhicule en quelques dizaines de millisecondes.

1.2 Architecture Générale du Système

Le projet repose sur **trois composants logiciels** distincts, qui communiquent entre eux selon le schéma suivant :



i Rôle de chaque composant

vision.py Capture la vidéo, détecte la main, classe le geste, publie la commande sur le broker MQTT.

bridge.py Souscrit au topic MQTT, reçoit la commande et la transmet à l'Arduino

via le port série USB.

motors.ino Lit la commande série et pilote le pont-H L298N qui alimente les moteurs.

1.3 Gestes Reconnus

Le tableau ci-dessous résume la correspondance geste → commande → mouvement.

Émoji	Geste	Commande MQTT	Mouvement
Poing	Poing fermé	stop	Arrêt complet
Main	5 doigts ouverts	avant	Avancer
Index	Index seul	gauche	Tourner à gauche
V	Index + Majeur	droite	Tourner à droite

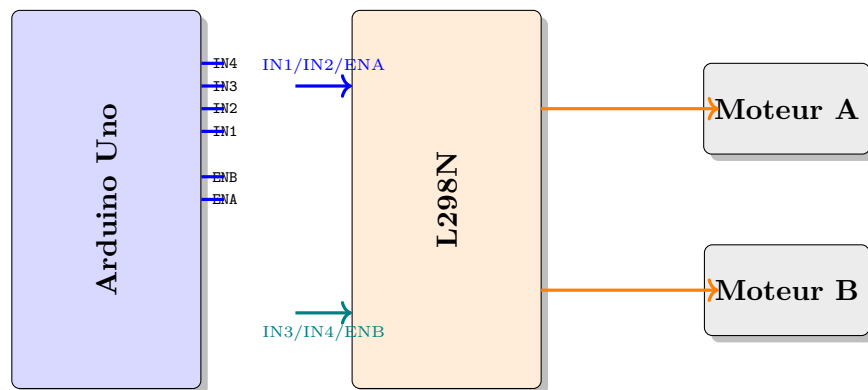
Chapitre 2

Matériel Nécessaire

2.1 Liste du Matériel

Composant	Modèle type	Rôle	Quantité
Microcontrôleur	Arduino Uno R3	Pilote les moteurs via L298N	1
Ordinateur/SBC	Raspberry Pi 4	Exécute les scripts Python	1
	(ou PC Linux/-Windows)		
Pont-H moteur	L298N	Interface entre Arduino et moteurs DC	1
Moteurs DC	Moteurs 3-6V TT	Entraînent les roues	2 ou 4
Châssis voiture	Kit 2WD/4WD	Structure mécanique	1
Webcam	USB 640x480 min	Capture la vidéo de la main	1
Câble USB	USB-A vers USB-B	Communication série Raspberry Pi ↔ Arduino	1
Alimentation	Batterie 7.4V LiPo ou 6xAA	Alimentation moteurs et carte	1
Câbles Dupont	Mâle/Femelle	Connexions entre composants	~20

2.2 Schéma de Câblage Arduino – L298N – Moteurs



Correspondance des broches

Pin Arduino	Broche L298N	Fonction
8	IN1	Direction moteur A
9	IN2	Direction moteur A
5	ENA	Vitesse moteur A (PWM)
10	IN3	Direction moteur B
11	IN4	Direction moteur B
6	ENB	Vitesse moteur B (PWM)
GND	GND	Masse commune

Important : Les pins ENA et ENB du L298N doivent être *débrochés* (jumper retiré) pour être contrôlables par PWM depuis l'Arduino.

Chapitre 3

Installation des Logiciels et Dépendances

3.1 Environnement côté Ordinateur / Raspberry Pi

3.1.1 Python 3 et pip

Python 3.8 ou supérieur est requis. Sur Raspberry Pi OS ou Ubuntu :

```
1 sudo apt update && sudo apt upgrade -y
2 sudo apt install python3 python3-pip python3-venv -y
```

Listing 3.1 – Installation de Python et pip

3.1.2 Création d'un environnement virtuel (recommandé)

```
1 python3 -m venv env_voiture
2 source env_voiture/bin/activate    # Linux / macOS
3 # Sur Windows : env_voiture\Scripts\activate
```

Listing 3.2 – Environnement virtuel Python

3.1.3 Bibliothèques Python requises

```
1 pip install opencv-python mediapipe paho-mqtt pyserial
```

Listing 3.3 – Installation des dépendances Python

Paquet	Rôle	Version testée
opencv-python	Capture vidéo, affichage, traitement image	4.8+
mediapipe	Détection et suivi de la main (landmarks)	0.10+
paho-mqtt	Client MQTT (publication/souscription)	1.6+
pyserial	Communication série avec l'Arduino	3.5+

3.2 Installation du Broker MQTT (Mosquitto)

```

1 sudo apt install mosquitto mosquitto-clients -y
2 sudo systemctl enable mosquitto
3 sudo systemctl start mosquitto
4
5 mosquitto_pub -h localhost -t test/commande -m "avant"
6 mosquitto_sub -h localhost -t test/commande

```

Listing 3.4 – Installation de Mosquitto

Configuration réseau

Si `vision.py` s'exécute sur une machine *différente* de `bridge.py`, remplacez "localhost" dans `vision.py` par l'adresse IP de la machine hébergeant Mosquitto (ex. "10.10.33.32"). Assurez-vous que le port **1883** est ouvert dans le pare-feu.

3.3 Environnement côté Arduino

1. Télécharger et installer l'**Arduino IDE** (version 2.x recommandée) : <https://www.arduino.cc/en/software>
2. Brancher l'Arduino Uno via USB.
3. Sélectionner le bon port série (/dev/ttyACM0 sous Linux, COM3 sous Windows, etc.).
4. Aucune bibliothèque tierce n'est nécessaire pour le sketch moteur.

3.3.1 Identifier le port série sous Linux

```

1 ls /dev/ttyACM*
2 ls /dev/ttyUSB*
3 dmesg | grep tty | tail -5

```

Listing 3.5 – Trouver le port série de l'Arduino

Chapitre 4

Code Source Commenté

4.1 Sketch Arduino : Contrôle des Moteurs (motors.ino)

```
1  int IN1 = 8,  IN2 = 9,  ENA = 5;
2  int IN3 = 10, IN4 = 11, ENB = 6;
3  int VITESSE = 130;
4
5  void setup() {
6      Serial.begin(9600);
7      pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT); pinMode(ENA, OUTPUT);
8      pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT); pinMode(ENB, OUTPUT);
9      arreter();
10 }
11
12 void loop() {
13     if (Serial.available()) {
14         String cmd = Serial.readStringUntil('\n');
15         cmd.trim();
16         if (cmd == "avant")  avancer();
17         else if (cmd == "stop")  arreter();
18         else if (cmd == "droite") tournerDroite();
19         else if (cmd == "gauche") tournerGauche();
20     }
21 }
22
23 void avancer() {
24     analogWrite(ENA, VITESSE); analogWrite(ENB, VITESSE);
25     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
26     digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
27 }
28
29 void arreter() {
30     analogWrite(ENA, 0); analogWrite(ENB, 0);
31     digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
32     digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
33 }
34
35 void tournerDroite() {
36     analogWrite(ENA, VITESSE); analogWrite(ENB, VITESSE);
```

```
37     digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
38     digitalWrite(IN3, LOW);   digitalWrite(IN4, HIGH);
39 }
40
41 void tournerGauche() {
42     analogWrite(ENA, VITESSE); analogWrite(ENB, VITESSE);
43     digitalWrite(IN1, LOW);   digitalWrite(IN2, HIGH);
44     digitalWrite(IN3, HIGH);  digitalWrite(IN4, LOW);
45 }
```

Listing 4.1 – Sketch Arduino complet – contrôle moteurs

Ajustement de la vitesse

La variable VITESSE (valeur PWM entre 0 et 255) contrôle la puissance envoyée aux moteurs. Commencez avec 100–130 puis augmentez progressivement.

4.2 Script Python : Pont MQTT – Série (bridge.py)

```
1 import paho.mqtt.client as mqtt
2 import serial
3 import time
4
5 ser = serial.Serial('/dev/ttyACM1', 9600, timeout=1)
6 time.sleep(2)
7
8 BROKER = "localhost"
9 TOPIC  = "test/commande"
10
11 def on_message(client, userdata, msg):
12     commande = msg.payload.decode().strip()
13     print(f"Recu : {commande}")
14     ser.write((commande + "\n").encode())
15
16 client = mqtt.Client()
17 client.on_message = on_message
18 client.connect(BROKER, 1883)
19 client.subscribe(TOPIC)
20
21 print("En attente de commandes...")
22 client.loop_forever()
```

Listing 4.2 – Script Python – Pont MQTT/Série

4.3 Script Python : Détection de Gestes (vision.py)

4.3.1 Configuration et paramètres

```
1 import cv2
2 import mediapipe as mp
3 import paho.mqtt.client as mqtt
4
5 BROKER = "10.10.33.32"
6 PORT   = 1883
7 TOPIC  = "test/commande"
8
9 client = mqtt.Client()
10 client.connect(BROKER, PORT)
11 client.loop_start()
12
13 mp_hands = mp.solutions.hands
14 mp_drawing = mp.solutions.drawing_utils
15 hands = mp_hands.Hands(
16     static_image_mode=False,
17     max_num_hands=1,
18     min_detection_confidence=0.85,
19     min_tracking_confidence=0.85
20 )
21
22 SEUIL_DOIGT_LEVE = 0.06
23 SEUIL_DOIGT_PLIE = 0.06
24 SEUIL_POUCE = 0.05
25 SEUIL_PAUME_OUVERTE = 4
26 NB_FRAMES_CONFIRM = 3
```

Listing 4.3 – Initialisation – vision.py

4.3.2 Détection de l'état des doigts

```
1 def etat_doigts(hand_landmarks):
2     lm = hand_landmarks.landmark
3     doigts = []
4     doigts.append(abs(lm[4].x - lm[3].x) > SEUIL_POUCE)
5     tips = [8, 12, 16, 20]
6     bases = [6, 10, 14, 18]
7     for tip, base in zip(tips, bases):
8         ecart = lm[base].y - lm[tip].y
9         doigts.append(ecart > SEUIL_DOIGT_LEVE)
10    return doigts
```

```

11
12 def est_poing_strict(hand_landmarks):
13     lm = hand_landmarks.landmark
14     tips = [8, 12, 16, 20]
15     bases = [6, 10, 14, 18]
16     for tip, base in zip(tips, bases):
17         if lm[base].y - lm[tip].y > SEUIL_DOIGT_PLIE:
18             return False
19     if abs(lm[4].x - lm[3].x) > SEUIL_POUCE:
20         return False
21     return True
22
23 def detecter_geste(hand_landmarks):
24     if est_poing_strict(hand_landmarks):
25         return "stop"
26     d = etat_doigts(hand_landmarks)
27     nb_leves = sum(d)
28     if nb_leves >= SEUIL_PAUME_OUVERTE:
29         return "avant"
30     if d[1] and not d[2] and not d[3] and not d[4]:
31         return "gauche"
32     if d[1] and d[2] and not d[3] and not d[4]:
33         return "droite"
34     return None
35
36 historique = []
37
38 def geste_confirme(nouveau_geste):
39     historique.append(nouveau_geste)
40     if len(historique) > NB_FRAMES_CONFIRM:
41         historique.pop(0)
42     if len(historique) == NB_FRAMES_CONFIRM and len(set(historique))
== 1:
43         return historique[0]
44     return None

```

Listing 4.4 – Fonctions de détection – vision.py

4.3.3 Boucle principale de capture et d'envoi

```

1 COULEURS = {
2     "stop" : (0, 0, 255),
3     "avant" : (0, 255, 0),
4     "droite" : (255, 165, 0),
5     "gauche" : (255, 0, 255),
6 }

```

```

7
8 cap = cv2.VideoCapture(0)
9 commande_precedente = None
10
11 while cap.isOpened():
12     success, image = cap.read()
13     if not success:
14         break
15     image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
16     resultats = hands.process(image_rgb)
17     image = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)
18     geste_brut = geste_final = None
19
20     if resultats.multi_hand_landmarks:
21         for hand_landmarks in resultats.multi_hand_landmarks:
22             mp_drawing.draw_landmarks(image, hand_landmarks,
23                                       mp_hands.HAND_CONNECTIONS)
24             geste_brut = detecter_geste(hand_landmarks)
25
26     geste_final = geste_confirme(geste_brut)
27
28     if geste_final and geste_final != commande_precedente:
29         client.publish(TOPIC, geste_final)
30         commande_precedente = geste_final
31
32     affichage = geste_final or geste_brut or "---"
33     couleur = COULEURS.get(affichage, (200, 200, 200))
34     cv2.putText(image, f"Geste : {affichage.upper()}",
35                 (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.2, couleur, 3)
36
37     if geste_brut:
38         progression = int((len(historique) / NB_FRAMES_CONFIRM) *
39 300)
40         cv2.rectangle(image, (10, 80), (10 + progression, 100),
41 couleur, -1)
42         cv2.rectangle(image, (10, 80), (310, 100), (255, 255, 255),
43 2)
44
45
46     cv2.imshow("Voiture Gesture Control", image)
47     if cv2.waitKey(5) & 0xFF == ord('q'):
48         break
49
50 cap.release()
51 cv2.destroyAllWindows()
52 client.loop_stop()
53 client.disconnect()

```

Listing 4.5 – Boucle principale – vision.py

Chapitre 5

Flux de Données et Protocoles

5.1 Le Protocole MQTT

MQTT (*Message Queuing Telemetry Transport*) est un protocole de messagerie *publish/subscribe* léger, idéal pour l'IoT. Son fonctionnement repose sur trois acteurs :

- Le **broker** (Mosquitto) : serveur central qui reçoit et redistribue les messages.
- Le **publisher** (`vision.py`) : publie une commande sur un *topic*.
- Le **subscriber** (`bridge.py`) : s'abonne au topic et réagit aux messages.

5.2 Communication Série (UART)

L'Arduino communique avec l'ordinateur via USB/UART à **9600 bauds**. `bridge.py` envoie une chaîne ASCII terminée par `\n`, que l'Arduino lit avec `Serial.readStringUntil('\n')`.

Chapitre 6

Mise en Œuvre et Démarrage

6.1 Ordre de Démarrage



Ordre impératif

Respectez impérativement cet ordre pour éviter les erreurs de connexion MQTT ou série.

1. Téléverser le sketch **Arduino** depuis l'Arduino IDE.

2. Démarrer le broker **Mosquitto** :

```
1 sudo systemctl start mosquitto
2 sudo systemctl status mosquitto
```

3. Lancer **bridge.py** :

```
1 source env_voiture/bin/activate
2 python3 bridge.py
```

4. Lancer **vision.py** :

```
1 source env_voiture/bin/activate
2 python3 vision.py
```

5. Présenter la **main** devant la webcam. Appuyer sur **q** pour quitter.

6.2 Résolution des Problèmes Courants

Symptôme	Cause probable	Solution
La webcam ne s'ouvre pas	Mauvais index de caméra	Essayez <code>cv2.VideoCapture(1)</code>
Gestes non détectés	Luminosité insuffisante	Améliorer l'éclairage
L'Arduino ne reçoit pas	Mauvais port série	Vérifier avec <code>ls /dev/ttyACM*</code>
Mosquitto refusé	Broker non démarré	<code>sudo systemctl start mosquitto</code>
Moteurs mauvais sens	Polarité inversée L298N	Inverser les câbles du moteur
La voiture ne bouge pas	VITESSE trop faible	Augmenter VITESSE, retirer jumpers
Gestes instables	Seuils trop permissifs	Augmenter SEUIL_DOIGT_LEVE

Chapitre 7

Pistes d'Amélioration

7.1 Améliorations Fonctionnelles

- **Marche arrière** : ajouter un geste (ex. pouce seul) et la fonction `reculer()` dans l'Arduino.
- **Vitesse variable** : encoder l'inclinaison du poignet pour moduler le PWM.
- **Télécommande sans fil** : remplacer la liaison série par un module ESP8266/ESP32.
- **Deux mains** : utiliser `max_num_hands=2` pour des contrôles combinés.

7.2 Améliorations Techniques

- **Sécurité MQTT** : activer l'authentification Mosquitto et TLS.
- **Démarrage automatique** : créer des services `systemd` pour les scripts.
- **Interface graphique** : ajouter un tableau de bord web (Flask + MQTT WebSocket).

Chapitre 8

Conclusion

Ce projet illustre de façon concrète l'intégration de plusieurs technologies complémentaires : la **vision par ordinateur** avec OpenCV et MediaPipe, la **communication IoT** légère avec MQTT/Mosquitto, et la **programmation embarquée** sur Arduino pour le contrôle matériel.

L'architecture en trois modules découplés (détection, pont, actionneur) est délibérément modulaire : chaque composant peut être modifié indépendamment, ce qui facilite l'expérimentation et l'extension du projet.



Pour aller plus loin

- Documentation MediaPipe Hands : <https://mediapipe.readthedocs.io>
- Documentation Paho-MQTT : <https://eclipse.dev/paho/clients/python/docs/>
- Guide L298N + Arduino : <https://www.arduino.cc/en/Tutorial/LibraryExamples>
- Mosquitto MQTT Broker : <https://mosquitto.org/documentation/>